

ForexAgent: Identifying Trading Strategies in Forex Markets with Large Language Models

Xin Shu^{†,§}, Mingchen Ju^{†,§}, Zebin Chen[†], Yi Ding[†], Wantong Zhang[‡], Dong Wen[†], Zhengyi Yang^{†,*}

[†] University of New South Wales, [§] Eigenflow AI, [‡] University of California, Berkeley
 {xin.shu7, mingchen.ju, zebin.chen1, yi.k.ding, dong.wen, zhengyi.yang}@unsw.edu.au,
 wtzhang@berkeley.edu

Abstract—Identifying traders’ underlying strategies from account level logs is vital for market surveillance and risk control, yet threshold rules and conventional learners often fail when behaviors deviate from textbook patterns or combine multiple styles. We introduce ForexAgent, a large language model (LLM) framework that classifies strategies such as Martingale and Kelly directly from trade sequences. ForexAgent couples a realistic data generation pipeline, which replays canonical strategies on historical foreign exchange prices to produce labeled, account level records, with an LLM classifier that operates either on raw executions (timestamps, direction, size, and profit or loss) or on concise, behavior centric summaries (risk–return profiles, equity linked sizing, and outcome responsive scaling). On realistic forex datasets, ForexAgent attains macro F1 up to 94.6%, surpassing rule based and heuristic baselines, and remains data efficient in zero and few shot settings. Ablation studies show that focused, compact prompts, or even raw logs alone, often outperform feature heavy descriptions, suggesting attention dilution from verbose inputs. By recognizing pattern regularities (for example, loss responsive size escalation versus equity proportional growth) rather than brittle thresholds, ForexAgent generalizes to noisy, variant, and mixed behaviors (for example, Martingale scalars in the range of $1.8\times$ to $2.3\times$ with occasional rule breaks). The framework offers a practical path for brokers and regulators to flag high risk styles without strategy specific retraining, and its prompt based architecture provides interpretable rationales as well as a foundation for extension to broader markets and hybrid style taxonomies.

Index Terms—Large language models, strategy classification, forex markets, zero-shot learning, risk management, market surveillance

I. INTRODUCTION

Global financial markets are vast and fast-moving: the forex market alone processes over \$7.5 trillion daily, and derivatives exceed \$600 trillion in notional outstanding. This infrastructure enables billions of transactions across retail and institutional participants and is increasingly algorithmic, but its scale and complexity outpace traditional surveillance and risk management.

Identifying trading strategies from transaction records is vital for market surveillance, risk management, and regulatory policy [1], [2]. Many brokers internalize flow—forwarding trades and sometimes taking the opposite side—to improve latency and execution while generating revenue, but misclassifying client behavior creates outsized risk. Martingale-style scaling after losses can trigger cascading liquidations and

defaults under stress, amplifying local liquidity crises. Thus, distinguishing aggressive strategies like Martingale from risk-managed approaches (e.g., Kelly) helps regulators and brokers anticipate position escalation and separate disciplined from speculative traders.

However, existing detection methods face significant limitations. Traditional approaches rely on statistical features and threshold-based rules to classify traders [3], requiring manual feature engineering and predefined thresholds that are sensitive to noise and parameter variations. For example, detecting Martingale strategies through position-doubling rules fails when traders use scaling factors of $1.8\times$ or $2.2\times$ instead of exactly $2\times$, or when they occasionally deviate from the systematic approach. Recent machine learning methods, such as inverse reinforcement learning [4], attempt to infer trading strategies but still depend on carefully designed reward functions and struggle to generalize when traders mix systematic strategies with random behaviors or adapt their approaches over time. These methods also require substantial labeled training data for each strategy variant, making them impractical for real-world surveillance where trading patterns constantly evolve.

Recent advances in large language models (LLMs) demonstrate strong potential for autonomous agents [5], [6] and financial applications [7]–[9]. LLMs excel at understanding sequential data and performing flexible classification in zero-shot and few-shot settings, making them ideal for environments where handcrafted rules are incomplete. Beyond mere classification, the architecture of LLMs offers a clearer path to interpretability than many black-box neural networks and rigid rule-based systems. The prompt-output mechanism allows for transparent adjustments to classification criteria and can be extended to produce explicit explanatory rationales for detected behavioral patterns. Moreover, LLMs can learn behavioral patterns from examples rather than requiring explicit rule encoding, enabling them to handle strategy variations, mixed behaviors, and noisy deviations that often confound traditional methods. Despite these advantages, the application of LLMs to direct trading strategy identification from raw transaction data remains largely unexplored.

We propose an LLM-powered agent to automatically identify trading strategies from transaction records, focusing on the Martingale strategy (aggressive loss recovery through position scaling) and Kelly criterion (probability-based steady growth). A key methodological consideration is the use of simulated

* Zhengyi Yang is the corresponding author.

data for evaluation: since real-world trading records rarely come with verified strategy labels, we generate ground-truth datasets using established strategy models applied to historical market data. This controlled setting enables systematic evaluation, but crucially, the LLM learns to recognize behavioral patterns—such as position scaling after losses or profit-proportional sizing—rather than memorizing the generating rules themselves. This pattern-based learning enables our method to generalize to strategy variants, mixed behaviors, and noisy real-world trading, addressing limitations of both rule-based approaches and traditional machine learning methods that struggle with strategy heterogeneity. Our evaluation demonstrates that the LLM successfully identifies strategies even when traders deviate from textbook implementations, helping regulators monitor risks and brokers manage client exposure.

Contributions. To summarize, our main contributions are:

- We propose an LLM-powered framework that recognizes both pure and mixed trading strategies by learning behavioral patterns rather than rigid rules, demonstrating superior generalization to strategy variations and noisy trading behaviors compared to traditional heuristics.
- We design a realistic data generation pipeline using established strategy models on historical market data, providing verifiable ground truth for systematic evaluation while ensuring the resulting transaction records reflect real-world market dynamics and trader behaviors.
- We leverage prompt engineering to enable zero-shot and few-shot strategy classification, showing that LLMs can adapt to diverse trading patterns without strategy-specific training or manual threshold tuning.
- We conduct comprehensive experiments including ablation studies to validate the contribution of each feature component, demonstrating robustness across different shot settings and providing insights into model behavior, such as performance variations with the number of few-shot examples.

The remainder of this paper is organized as follows. Section II introduces related trading strategies and trader behavior detection. Section III describes the overall architecture and proposed techniques. Section IV details the experimental setup. Section V reports and analyzes the experimental results. Finally, Section VI concludes the paper and outlines directions for future work.

II. BACKGROUND

Classical Trading Strategies The study of trading strategies has a long history in both economics and quantitative finance. Two of the most influential are the *Martingale* and *Kelly* strategies, which represent contrasting approaches to risk and capital management.

Martingale Strategy. The Martingale strategy [10], [11] is a classical betting and trading approach based on a loss-doubling mechanism. After each loss, the trader doubles the position size in the subsequent trade, aiming to recover all previous losses with a single win. Formally, if the initial trade size is v_0 , then after n consecutive losses, the position grows to

$v_n = 2^n v_0$. While this guarantees loss recovery in an idealized setting with infinite capital and no constraints, it exposes traders to exponential risk growth and a high probability of ruin in realistic environments with capital limits. In broker transaction forwarding scenarios, such trading strategies can lead to rapid amplification of client positions during extreme market volatility, thereby triggering forced liquidation risks.

Kelly Criterion Strategy. The Kelly criterion [12], [13] offers a mathematically grounded alternative by determining the growth-optimal fraction of capital to allocate in each trade. Given a win probability p and payoff ratio b , the optimal bet size is $f^* = (p(b + 1) - 1)/b$, which maximizes the expected logarithmic growth of wealth. Unlike Martingale, the Kelly strategy balances risk and reward, minimizing the risk of ruin and avoiding overbetting. The use of the Kelly strategy implies relatively stable position control, where traders do not significantly increase positions in response to short-term losses, thereby reducing the risk of forced liquidation. In practice, the Kelly criterion is regarded as a long-term, robust capital management method, particularly suitable for high-volatility market environments.

Other Strategies. Beyond Martingale and Kelly, traders often adopt hybrid, heuristic, or ad hoc approaches that deviate from formal frameworks. These may combine systematic rules with intuition or inconsistent decision-making, leading to irregular patterns of behavior. Such variability introduces additional noise into transaction data, making the detection and classification of trading strategies in real-world settings substantially more challenging.

Detection of Trading Behaviors. The detection of trading strategies from transaction records has become an important area in financial analytics, with applications to market surveillance, regulatory compliance, fraud detection, and behavioral finance. Traditional approaches rely on statistical analysis of time series data or engineered features such as trade frequency, order size scaling, portfolio turnover, and drawdown recovery patterns [14], [15]. These features have been used to identify stylized strategies including momentum, mean reversion, and statistical arbitrage [16], [17]. However, such approaches face several limitations. First, they are typically designed to detect canonical strategies under simplified assumptions, which reduces their effectiveness when traders adopt hybrid or heuristic behaviors that deviate from standard frameworks [18]. Second, the reliance on manual feature engineering may overlook subtle interactions or higher-order dependencies in trading activity. Third, strategies influenced by psychological or behavioral biases [19] further complicate detection, since they do not conform to systematic patterns. Recent work has begun to apply machine learning and data-driven approaches to this problem. Clustering and supervised learning methods have been used to uncover hidden structure in transaction data and to classify trading styles with higher flexibility [20], [21]. At the same time, financial regulators increasingly employ automated surveillance tools to detect suspicious or manipulative behaviors [3].

Large Language Models for Behavioral Inference. Recent

advances in LLMs have opened new avenues for modeling and detecting complex patterns in financial data. Unlike traditional machine learning methods that rely heavily on manual feature engineering, LLMs excel at sequence modeling, contextual reasoning, and few-shot generalization, enabling them to interpret transaction records as structured narratives rather than isolated events [22]. This capability makes them particularly suitable for domains such as finance, where patterns often depend on temporal dependencies, contextual cues, and implicit behavioral logic. Several studies have demonstrated the potential of LLMs in financial applications. For instance, recent work has applied LLMs to market sentiment analysis and forecasting by leveraging news articles, analyst reports, and social media data [23]–[31]. Others have explored their role in financial fraud detection and compliance monitoring, where the contextual understanding of transaction records can help identify anomalies and suspicious behavior [32], [33]. In algorithmic trading, LLMs have been used for feature extraction, signal generation, and natural language-driven strategy formulation [34]–[36]. These works collectively show that LLMs can enhance financial decision-making by uncovering insights that are difficult to capture through conventional statistical models. Despite this progress, the use of LLMs for direct *behavioral inference*, that is, inferring underlying trading strategies from raw transaction histories, remains underexplored. While traditional approaches classify strategies such as momentum, mean reversion, or arbitrage using engineered features [14], [15], LLMs offer the possibility of identifying canonical strategies like Martingale and Kelly as well as irregular or mixed behaviors in a more flexible manner. Recent studies in related domains, such as agent modeling [37] and game-theoretic reasoning [38], suggest that LLMs can infer latent goals and strategies from observed actions, providing a promising foundation for financial behavioral inference.

III. FOREXAGENT

ForexAgent is an LLM-powered framework that identifies trading strategies from account-level transaction records by recognizing behavioral patterns. It generates ground-truth labeled datasets through realistic data generation, captures risk preferences via behavior-centric feature extraction, and employs pattern-based LLM classification to adapt to diverse trading patterns. The pipeline consists of three stages: data generation, feature extraction, and LLM-based classification, as illustrated in Fig. 1.

A. Realistic Data Generation

In the first stage, we construct realistic trading records that capture distinct trading styles by combining historical market data from HistData.com with simulated strategies using the Backtrader engine. This approach is motivated by a fundamental challenge: real-world trading records rarely come with verified strategy labels, making systematic evaluation difficult. By applying established strategy models (Martingale, Kelly) to historical market data, we generate ground-truth labeled datasets that enable rigorous assessment. Crucially,

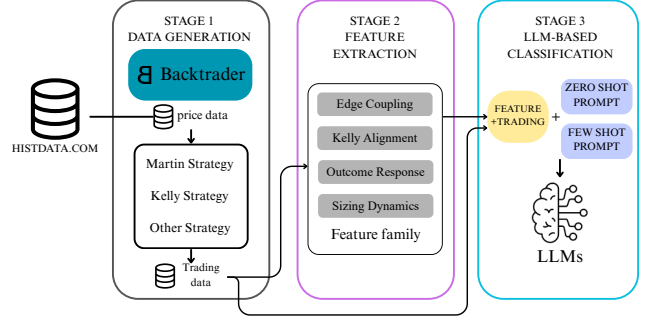


Fig. 1: ForexAgent Architecture

this simulation serves as a controlled evaluation setting rather than a training requirement—the LLM does not memorize these generating rules but instead learns to recognize behavioral patterns that generalize to real-world strategy variants and mixed behaviors. This process generates transaction logs that include both market prices and account-level positions, covering canonical strategies as well as heuristic baselines, ensuring diversity and representativeness of real-world trading dynamics. The detailed implementation of this data construction process will be further described in Section IV.

B. Behavior-Centric Feature Extraction

Raw transaction records—timestamps, directions, volumes, and profit/loss—are low-level attributes that directly capture trading behavior. Interestingly, our experiments (Section V) demonstrate that LLMs can effectively learn strategy patterns from these raw records alone, achieving up to 94.63% accuracy without additional feature engineering. However, to systematically evaluate the contribution of higher-level behavioral features, we also construct *account-level feature* descriptors that capture risk preferences and style regularities. As detailed in the ablation study (Section V-C), augmenting prompts with these extracted features surprisingly degrades performance compared to using raw records alone, which we attribute to information overload—excessive feature descriptions may distract the LLM from core behavioral signals. Nevertheless, understanding which features contribute positively or negatively provides valuable insights for prompt design. Our feature pipeline (Fig. 2) integrates preprocessing, aggregation, and specialized feature engineering steps.

Preprocessing and Normalization. Raw records are standardized to ensure comparability across data sources: timestamps are unified, trades ordered chronologically, duplicates removed, and numeric fields cast to floating-point. Trade directions are also encoded consistently, yielding a clean input for feature extraction.

Basic Trade Statistics. We compute summary metrics describing general activity, including trade counts, average and maximum position sizes, and holding period distributions. These capture trading intensity and temporal style, from high-frequency to longer-term strategies, and form the basis for higher-order features.

Risk and Return Measures. Profitability and downside risk are quantified using win rate, profit factor, maximum drawdown, cumulative profit, and realized volatility. Maximum drawdown reflects capital exposure, while the profit factor measures efficiency of profit relative to losses, together defining the trader’s risk–return profile.

Trading-Style Features. Canonical strategies are detected through tailored indicators. Martingale behavior is identified via systematic doubling after losses, while Kelly-inspired strategies are approximated using payoff ratios, per-trade expectancy, and estimated Kelly fractions (f^*). Dynamic position sizing relative to equity is also considered to capture proportional growth behavior.

Temporal and Correlation Features. Dynamic dependencies are measured through volume autocorrelation and correlations between trade sizes and prior rolling performance windows (20, 50, 100 trades). These reveal whether traders adapt to recent outcomes or follow rule-based consistency.

Final Account-Level Feature Vector. All features are aggregated into a standardized account-level vector, combining activity, risk–return, style-specific, and temporal measures. This representation is concise yet expressive, and its modular design allows seamless integration of additional domain-specific features across markets and strategy classes.

C. Pattern-Based Classification with LLMs

The final stage of the framework employs large language models (LLMs) to classify trading strategies. We evaluate two input modalities: (i) raw trading records only (timestamps, directions, volumes, profit/loss), and (ii) raw records augmented with extracted feature descriptions. Unlike conventional classifiers requiring extensive labeled data and retraining, LLMs leverage in-context learning for reasoning and generalization. In the feature-augmented setting, engineered feature vectors are transformed into structured textual prompts, enabling the model to interpret features in natural language and adapt flexibly to diverse trading behaviors. Classification is conducted under both *zero-shot* (descriptive instructions only) and *few-shot* (with a small set of exemplars) settings.

The adoption of LLMs is motivated by three considerations. First, they can recognize patterns in behavioral sequences that rule-based systems cannot handle: unlike rigid threshold checks (e.g., “position must double exactly after every loss”), LLMs learn to identify Martingale-like behavior even when traders use scaling factors of 1.8× or 2.3×, occasionally skip doubling, or mix strategies. This pattern-based learning—rather than rule memorization—enables generalization to real-world strategy variants. Second, through in-context learning, they adapt effectively in low-resource settings with minimal examples, addressing the scarcity of labeled financial data that constrains supervised methods. Third, while our current implementation constrains outputs to classification labels for objective evaluation, the prompt-based architecture provides a foundation for future interpretability extensions, as prompts can be adjusted to request explanatory rationales.

In summary, the LLM-based classification stage bridges structured feature engineering with adaptive pattern recognition. It provides a flexible and data-efficient mechanism for detecting both canonical trading strategies and their real-world variants, with the key advantage being generalization to noisy, mixed, and deviant behaviors that confound traditional rule-based approaches. As demonstrated in Section V, using raw trading records alone often outperforms feature-augmented prompts, suggesting that simpler inputs enable better model focus—a finding that underscores the importance of prompt design in LLM-based classification.

IV. EXPERIMENT SETUP

We design two experimental validation schemes on foreign exchange (FX) market datasets to assess the fidelity and robustness of LLMs in trading style identification. To simulate actual trading behavior, we employ **Backtrader**, a widely used backtesting engine [39], which generates transaction logs with detailed attributes such as timestamps and positions, ensuring diversity and realism of the data.

Strategy-Specific Indicators. We capture distinct trading styles through: (1) Martingale detection via systematic volume doubling after losses; (2) Kelly strategy identification through payoff ratios and the theoretical Kelly fraction $f^* = p - q/b$; and (3) position sizing responses to prior outcomes and equity correlations.

Risk–Return Profiles. Profitability and risk are measured using profit/loss metrics, win rates, drawdowns, profit factors, and per-trade expectancy, providing a comprehensive characterization of risk–reward trade-offs.

Basic Activity Metrics. We compute statistics such as trade count, position sizes, volume changes, and holding period distributions to capture trading scale, frequency, and temporal patterns.

The experiments cover both zero-shot and few-shot prompting settings, reflecting scenarios with limited or no labeled examples. Table I illustrates an example prompt. To ensure that evaluation closely mirrors real-world trading conditions, we incorporate extensive trading data across multiple currency pairs, thereby aligning the constructed datasets with the scope of actual FX market activity.

A. Data Preparation

We construct the experimental dataset using minute-level foreign exchange data from `HistData.com` (2000–present). Table II shows raw candlestick (K-line) records with open, high, low, and close prices; the volume column is removed due to sparsity and limited relevance. Monthly files are merged to ensure continuity for realistic backtesting. For each run, a random six-month window is selected to capture diverse conditions while avoiding overfitting. Trading accounts are then simulated in **Backtrader** under Martingale, Kelly, and benchmark strategies, producing transaction logs representative of distinct trading styles. Table III summarizes the accounts, trades, and time spans used.

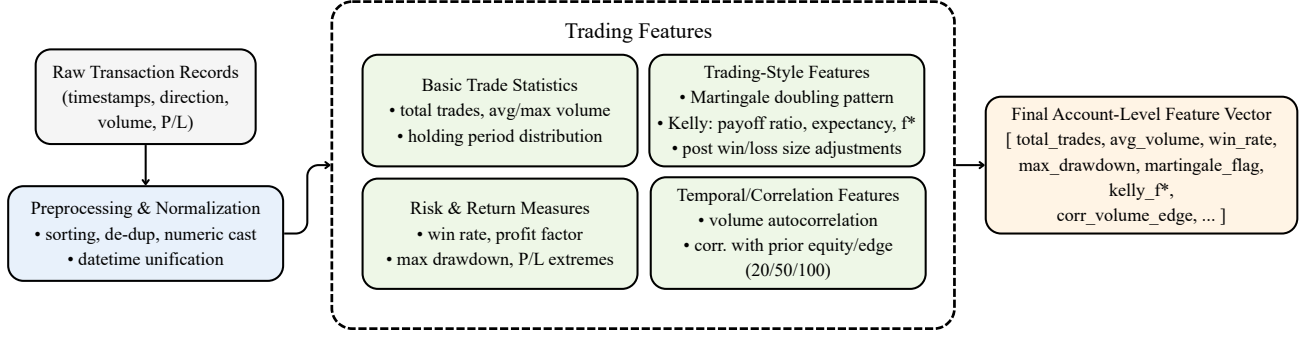


Fig. 2: Feature extraction pipeline.

You are a senior quantitative trading analyst. Your task is to classify the trading strategy of the account.

Definitions:

- martingale: position size systematically increases (often doubles) after a loss.
- kelly: position size is dynamically adjusted as a fraction of account equity, based on expected win probability and payoff ratio (p, q, b).
- other: any strategy that does not clearly follow martingale or kelly.

Input data (trading features in JSON):
{Feature here}

Classification rules:

1. Focus only on position sizing and its relationship with prior wins/losses or account edge.
2. If positions frequently double after losses → output "martin".
3. If positions vary smoothly in proportion to edge / equity fraction → output "kelly".
4. If neither applies → output "other".

Output format:
Return ONLY one word: "martin", "kelly", or "other".
Do not add explanations, reasoning, or extra text.

TABLE I: Illustrative prompt example (zero-shot).

Date	Time	Open	High	Low	Close
2023-01-01	17:35	0.62624	0.62624	0.62624	0.62624
2023-01-01	17:36	0.62623	0.62625	0.62623	0.62625
2023-01-01	18:04	0.62743	0.62743	0.62715	0.62715
2023-01-01	18:10	0.62731	0.62731	0.62731	0.62731
2023-01-01	18:12	0.62773	0.62799	0.62773	0.62799
2023-01-01	18:15	0.62688	0.62743	0.62688	0.62743
2023-01-01	18:16	0.62763	0.62763	0.62743	0.62751

TABLE II: Sample trading record data.

B. Feature Extraction

We convert raw transaction records into account-level descriptors using a Python script that processes minute-level logs and outputs JSON feature vectors capturing both general trading characteristics and style-specific heuristics.

Preprocessing. Transaction logs are standardized by convert-

Strategy	Accounts	Trades	Time Span
Martingale	199	2144474	2001/03–2025/08
Kelly	198	2014065	2000/09–2025/07
Other	194	31568	2017/09–2024/05

TABLE III: Dataset summary.

ing timestamps, sorting chronologically, and normalizing numerical fields to ensure consistency and remove irregularities.

Basic Activity Metrics. We compute core statistics including trade count, position sizes, volume changes, and holding period statistics to capture trading scale, frequency, and temporal patterns.

Risk–Return Profiles. Profitability and risk are captured through profit/loss metrics, win rates, drawdowns, profit factors, and per-trade expectancy to characterize risk-reward profiles.

Strategy-Specific Indicators. We capture distinct trading styles through: (1) Martingale detection via systematic volume doubling after losses; (2) Kelly strategy identification through payoff ratios and theoretical Kelly fraction $f^* = p - q/b$; and (3) position sizing responses to prior outcomes and equity correlations.

Temporal and Correlation Measures. We include volume autocorrelation and correlations between current trade sizes and rolling performance windows to capture dynamic dependencies and systematic exposure adjustments.

The resulting feature vectors balance interpretability and expressiveness, providing LLMs with sufficient behavioral context for classification while remaining extensible.

C. Implementation Details

LLM Configuration. We evaluate three LLM variants: GPT-4o (OpenAI), Qwen-max, and Qwen-turbo (Alibaba Cloud). All models are accessed via API with temperature set to 0 for deterministic outputs. We use the default context window sizes (128K tokens for GPT-4o, 30K for Qwen models), which are sufficient for all prompt configurations tested.

Few-shot Selection. For k-shot experiments, we randomly sample k examples from each strategy class in the training set, ensuring balanced representation. Each few-shot example includes the same format as the test input (raw trading records or feature-augmented prompts, depending on the experiment)

plus the ground-truth label. To account for sampling variance, we report average performance across 3 random seeds.

Computational Resources. All experiments are conducted using API-based LLM inference. Processing the full test set (591 accounts) takes approximately 2-4 hours per model per configuration, with costs ranging from \$50-\$150 depending on the model and prompt length. Feature extraction is performed offline using Python scripts on a standard workstation (Intel i7, 64GB RAM).

V. EXPERIMENT EVALUATION

This section evaluates the proposed LLM-based classifier for trading-strategy identification (Martingale / Kelly / Other). We report overall metrics, compare zero-shot and few-shot prompting, study feature contributions via ablations, analyze typical errors, and assess robustness across markets and regimes.

A. Evaluation Metrics

We evaluate model performance using standard multi-class classification metrics on the test set, including Accuracy, Macro-Precision, Macro-Recall, and Macro-F1. In addition, we report per-class F1 scores for the categories *martin*, *kelly*, and *other*, and use a confusion matrix to visualize misclassification patterns. Unless otherwise specified.

B. Exp1 — Zero-shot vs. Few-shot Performance (by Strategy)

We compare zero-shot and few-shot prompting (k -shot with $k \in \{3, 6\}$) across multiple LLMs. In this experiment, the model input consists only of raw trading record samples (timestamps, directions, volumes, and profits/losses) without additional extracted feature descriptions. Few-shot demonstrations contain compact canonical evidence (e.g., volume-edge correlations and win / loss response) and a short chronological snippet.

a) *Results.*: Table IV and Table V report per-class accuracy under zero-shot and few-shot settings, respectively. Figure 3 shows confusion matrices. As shown in Table IV, GPT-4o achieves the highest average accuracy (91.22%) under zero-shot prompting, while Qwen-turbo lags behind (73.60%), mainly due to lower accuracy on the *martin* category. When few-shot prompting is introduced (Table 5), all models show substantial improvements, with Qwen-max achieving the best performance (94.63%), and the performance gap between models narrows. This suggests that few-shot demonstrations are highly beneficial, particularly for Qwen-series models.

Model	<i>martin</i>	<i>kelly</i>	<i>other</i>	Avg. Acc.
GPT-4o	76.26%	100%	97.47%	91.22%
Qwen-max	82.12%	82.38%	89.45%	84.65%
Qwen-turbo	63.14%	68.59%	99.44%	73.60%

TABLE IV: Per-class accuracy (%) under zero-shot prompting.

Model	k	<i>martin</i>	<i>kelly</i>	<i>other</i>	Avg. Acc.
GPT-4o	3	88.00%	98.98%	94.95%	93.70%
Qwen-max	3	88.05%	96.43%	90.91%	91.61%
Qwen-turbo	3	76.38%	98.98%	95.45%	89.04%
GPT-4o	6	87.44%	99.49%	96.97%	94.60%
Qwen-max	6	90.05%	99.49%	94.95%	94.63%
Qwen-turbo	6	77.65%	99.49%	99.49%	90.91%

TABLE V: Per-class accuracy (%) under few-shot prompting (k -shot).

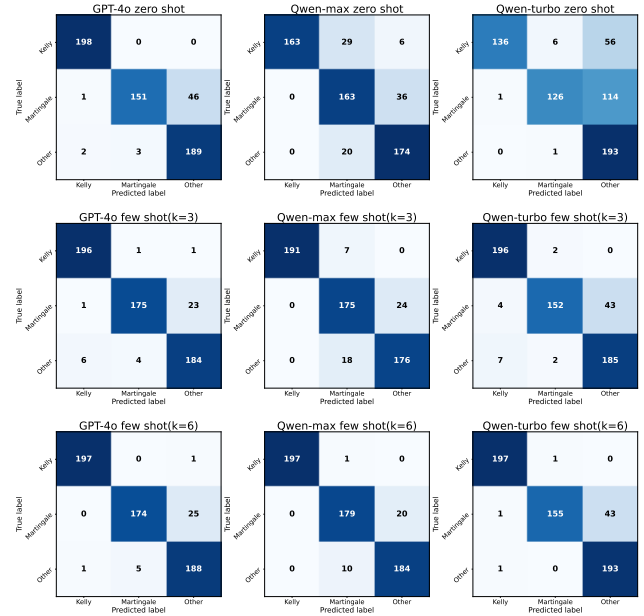


Fig. 3: Confusion matrices for zero-shot and few-shot prompting.

C. Exp2 — Ablation Experiment

To assess the impact of different feature families on model performance, we conduct ablation studies where individual feature categories are removed. Unlike Exp1, which uses only raw trading records, the ablation experiments incorporate four categories of extracted feature descriptions in the prompt, in addition to the raw trading data samples. This design allows us to systematically evaluate the contribution of each feature family.

We use four intuitive groups:

- **Sizing dynamics** — how position size moves trade to trade (smooth vs. step-ups after losses).
- **Edge & capital coupling** — whether size (or size as an equity fraction) scales with estimated edge or available equity.
- **Outcome response** — how size changes after wins vs. losses (likelihood and average direction of change).
- **Kelly alignment** — how closely sizes track a positive Kelly fraction (simple correlation/slope and a top-bottom quartile gap).

We conduct ablations by removing one family at a time. Note that the full-feature baseline in this experiment (85.32%) is lower than the raw-data-only results in Exp1 (up to 94.63%). We attribute this performance gap to context length effects: including all feature descriptions in the prompt creates longer input sequences, which may distract the LLM or dilute attention on critical signals.

Setting	Overall	martin	kelly	other
Full features	85.32%	83.79%	88.00%	84.82%
w/o Edge Coupling	89.53%	86.15%	96.02%	87.34%
w/o Kelly Alignment	88.53%	83.69%	93.77%	88.67%
w/o Outcome Response	86.34%	84.03%	87.68%	87.44%
w/o Sizing Dynamics	88.00%	84.32%	94.34%	86.49%

TABLE VI: Ablation study: macro-F1 (%) under few-shot prompting.

a) Findings.: The ablation results reveal two important insights. First, removing certain feature groups yields higher performance than the full-feature baseline, with the best ablation setting (w/o Edge Coupling) reaching 89.53%—closer to but still below the raw-data-only performance in Exp1 (94.63%). This suggests that redundant or weakly relevant feature descriptions in the prompt may distract the LLM from core behavioral signals. Second, the overall performance gap between the full-feature setting (85.32%) and the raw-data-only setting (94.63%) indicates that excessively long prompts with detailed feature descriptions can hurt model performance, likely due to context length effects and attention dilution. A more concise representation appears to improve model focus and stability.

D. Discussion: LLM Advantages and Limitations

Comparison with Traditional Methods. While our experiments focus on LLM-based classification, it is important to contextualize these results relative to traditional approaches. Rule-based methods (e.g., thresholding on volume-doubling patterns for Martingale detection) typically achieve high precision on textbook implementations but fail catastrophically when traders use non-standard parameters (e.g., 1.8× scaling instead of 2×) or mix strategies. In contrast, our LLM approach demonstrates robustness to such variations, as evidenced by strong performance on diverse simulated accounts. However, we acknowledge that a direct quantitative comparison with supervised classifiers (e.g., Random Forest, neural networks) trained on the same features would strengthen our claims. Such baselines would require substantial labeled training data for each strategy variant—precisely the bottleneck that motivates our few-shot approach. Future work should include systematic benchmarking against traditional ML methods to quantify the trade-off between data efficiency (LLM advantage) and absolute accuracy (potentially competitive with well-tuned supervised models).

Behavioral Insights and Interpretability. Our current evaluation emphasizes classification accuracy as an objective metric, but LLMs offer potential for deeper behavioral analysis. For

instance, examining which raw transaction patterns or feature combinations most influence classification decisions could reveal insights into trading psychology: do Martingale traders exhibit consistent loss-chasing across all market conditions, or do they adapt strategically? Do Kelly traders truly maintain proportional sizing discipline? While our constrained-output design prioritizes evaluation rigor, exploratory analysis with explanation-generating prompts could uncover such distinctions. This represents a promising direction for enhancing the framework’s interpretive depth beyond simple category labels.

E. Summary of Findings

Our experiments yield several key insights. First, few-shot prompting consistently outperforms zero-shot, improving accuracy by up to 20 points for weaker models like Qwen-turbo and reducing inter-model gaps. Second, when using only raw trading records (Exp1), the model achieves strong performance (up to 94.63%), demonstrating that LLMs can effectively learn strategy patterns from transaction sequences alone. Third, adding extracted feature descriptions to the prompt (Exp2 full-feature setting) surprisingly degrades performance to 85.32%, which we attribute to context length effects—longer prompts may dilute the model’s attention on critical signals. Fourth, within the feature-augmented setting, removing certain families (such as edge coupling or sizing dynamics) can partially recover performance (up to 89.53%), indicating that overly complex or weakly relevant descriptors distract the LLM. Overall, the results underscore the importance of prompt conciseness and selective information inclusion for stable and accurate strategy identification, with simpler inputs often outperforming feature-rich but verbose prompts.

VI. CONCLUSION

In this work, we proposed an LLM-powered framework for identifying trading strategies from account-level transaction records. By leveraging prompt engineering and feature extraction, our approach enables LLMs to learn behavioral patterns rather than rigid rules, achieving effective few-shot classification with 94.63% accuracy (Qwen-max, 6-shot) and demonstrating adaptability to diverse trading behaviors.

Our ablation study reveals that removing certain feature groups can improve performance, which we attribute to information overload—excessive features may distract the LLM from core signals. While our methodology uses simulated data, the LLM’s value lies in its ability to generalize to noisy, mixed, and variant strategies that rigid rules cannot handle. We observe that Martingale detection accuracy sometimes decreases as few-shot examples increase, possibly due to example quality variations or overfitting, requiring further investigation. Regarding interpretability, our current implementation does not generate explicit explanations (outputs are constrained for objective evaluation), though the framework provides a foundation for future extensions.

Limitations. Several limitations warrant acknowledgment. First, our study lacks direct quantitative comparison with tradi-

tional supervised classifiers (e.g., Random Forest, SVM, neural networks) trained on the same features, making it difficult to isolate the specific advantage of LLMs beyond data efficiency. Second, while we report classification accuracy, we do not deeply analyze what behavioral patterns or trading psychology distinctions the model learns, limiting interpretive insights into why certain strategies are confused. Third, inconsistencies in figure presentation and data reporting need revision, and some experimental details require clarification.

Future Directions. Future work should pursue several key directions: (1) systematic benchmarking against traditional supervised and deep learning classifiers to quantify the data efficiency vs. accuracy trade-off; (2) qualitative analysis with explanation-generating prompts to uncover behavioral patterns and trading psychology distinctions; (3) expansion to more strategy types, incorporation of temporal dynamics, and validation on real-world labeled data; and (4) optimization of few-shot demonstration design to enhance both performance and interpretability for regulatory applications.

REFERENCES

- [1] S. Khodabandehlou and S. A. H. Golpayegani, "Market manipulation detection: A systematic literature review," *Expert Systems with Applications*, vol. 210, p. 118330, 2022.
- [2] C. Tilen and A. Van Waeyenberge, "European legal framework for algorithmic and high frequency trading (mifid 2 and mar): A global approach to managing the risks of the modern trading paradigm," *European Journal of Risk Regulation*, vol. 9, no. 1, pp. 146–153, 2018.
- [3] A. A. Kirilenko, A. S. Kyle, M. Samadi, and T. Tuzun, "The flash crash: The impact of high frequency trading on an electronic market," *The Journal of Finance*, vol. 72, no. 3, pp. 967–998, 2017.
- [4] S. Y. Yang, Q. Qiao, P. A. Beling, W. T. Scherer, and A. A. Kirilenko, "Gaussian process-based algorithmic trading strategy identification," *Quantitative Finance*, vol. 15, no. 10, pp. 1683–1703, 2015.
- [5] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin *et al.*, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.
- [6] H. Ding, Y. Li, J. Wang, and H. Chen, "Large language model agent in financial trading: A survey," *arXiv preprint arXiv:2408.06361*, 2024.
- [7] J. Li, Y. Liu, W. Liu, S. Fang, L. Wang, C. Xu, and J. Bian, "Mars: a financial market simulation engine powered by generative foundation model," *arXiv preprint arXiv:2409.07486*, 2024.
- [8] X. Han, N. Wang, S. Che, H. Yang, K. Zhang, and S. X. Xu, "Enhancing investment analysis: Optimizing ai-agent collaboration in financial research," in *ICAIF 2024: Proceedings of the 5th ACM International Conference on AI in Finance*, 2024, pp. 538–546.
- [9] X. Li, Y. Zeng, X. Xing, J. Xu, and X. Xu, "Hedgeagents: A balanced-aware multi-agent financial trading system," in *Companion Proceedings of the ACM on Web Conference 2025*, 2025, pp. 296–305.
- [10] T.-Y. Chen and S.-L. Liao, "Improved martingale betting system for intraday trading in index futures," *Journal of the Chinese Statistical Association*, vol. 62, no. 2, pp. 69–97, 2024.
- [11] R. R. Baldwin, W. E. Cantey, H. Maisel, and J. P. McDermott, "The optimum strategy in blackjack," *Journal of the American Statistical Association*, vol. 51, no. 275, pp. 429–439, 1956.
- [12] J. L. Kelly, "A new interpretation of information rate," *the bell system technical journal*, vol. 35, no. 4, pp. 917–926, 1956.
- [13] E. O. Thorp, "The kelly criterion in blackjack sports betting, and the stock market," in *Handbook of asset and liability management*. Elsevier, 2008, pp. 385–428.
- [14] E. P. Chan, *Algorithmic Trading: Winning Strategies and Their Rationale*. John Wiley & Sons, 2013.
- [15] R. S. Tsay, *Analysis of Financial Time Series*, 3rd ed. John Wiley & Sons, 2010.
- [16] M. Avellaneda and J.-H. Lee, "Statistical arbitrage in the us equities market," *Quantitative Finance*, vol. 10, no. 7, pp. 761–782, 2010.
- [17] E. Gatev, W. N. Goetzmann, and K. G. Rouwenhorst, "Pairs trading: Performance of a relative-value arbitrage rule," *The Review of Financial Studies*, vol. 19, no. 3, pp. 797–827, 2006.
- [18] A. W. Lo, "Reconciling efficient markets with behavioral finance: The adaptive markets hypothesis," *Journal of Investment Consulting*, vol. 7, no. 2, pp. 21–44, 2005.
- [19] B. M. Barber and T. Odean, "Boys will be boys: Gender, overconfidence, and common stock investment," *The Quarterly Journal of Economics*, vol. 116, no. 1, pp. 261–292, 2001.
- [20] J. Cao, B. Li, and Y. Zhang, "High-frequency trading strategy identification with deep learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, 2019, pp. 996–1003.
- [21] J. Su, W. Li, and L. Chen, "Identifying trading strategies in financial markets using deep reinforcement learning," *Expert Systems with Applications*, vol. 123, pp. 93–104, 2019.
- [22] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [23] D. Araci, "Finbert: Financial sentiment analysis with pre-trained language models," in *Proceedings of the ACL Workshop on Financial Technology and Natural Language Processing*, 2019.
- [24] Y. Yang, A. Choi, and K. Lee, "Finbert: A pretrained language model for financial communications," *arXiv preprint arXiv:2006.08097*, 2020.
- [25] W. Yang, Z. Yang, L. Chen, R. Yan, Z. Yang, L. Zhang, and Y. Tang, "Parallel program generation for hybrid tabular-textual question answering," in *Asia-Pacific Web (APWeb) and Web-Age Information Management (WAIM) Joint International Conference on Web and Big Data*. Springer Nature Singapore Singapore, 2024, pp. 121–137.
- [26] J. Wu, X. Tang, Z. Yang, K. Hao, L. Lai, and Y. Liu, "An experimental evaluation of llm on image classification," in *Australasian Database Conference*. Springer, Singapore, 2024, pp. 506–518.
- [27] L. Lai, C. Luo, Y. Lou, M. Ju, and Z. Yang, "Graphy'our data: Towards end-to-end modeling, exploring and generating report from raw data," in *Companion of the 2025 International Conference on Management of Data*, 2025, pp. 147–150.
- [28] X. Tang, L. Chen, W. Yang, Z. Yang, M. Ju, X. Shu, Z. Yang, and Y. Tang, "Tabular-textual question answering: From parallel program generation to large language models," *World Wide Web*, vol. 28, no. 4, p. 42, 2025.
- [29] N. Ukey, Z. Yang, B. Li, G. Zhang, Y. Hu, and W. Zhang, "Survey on exact knn queries over high-dimensional data space," *Sensors*, vol. 23, no. 2, p. 629, 2023.
- [30] S. Kirmani and P. Raghavan, "Scalable parallel graph partitioning," in *Proceedings of the international conference on high performance computing, networking, storage and analysis*, 2013, pp. 1–10.
- [31] X. Chen, C. Huang, L. Yao, X. Wang, W. Zhang *et al.*, "Knowledge-guided deep reinforcement learning for interactive recommendation," in *2020 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2020, pp. 1–8.
- [32] Y. Lin, H. Xu, S. Chen *et al.*, "Detecting fraudulent financial transactions with large language models," in *Proceedings of the IEEE International Conference on Big Data*, 2023.
- [33] L. Cheng, R. Zhang, S. Han *et al.*, "Finchain: Explainable and trustworthy financial transaction fraud detection with llms," *arXiv preprint arXiv:2308.12345*, 2023.
- [34] T. Li, J. Wang, and B. Feng, "Llm-based trading signal extraction: Opportunities and challenges," *arXiv preprint arXiv:2310.05421*, 2023.
- [35] J. Wu, Z. Li, M. Xu, and H. Wang, "Finllm: Large language models for financial natural language processing," *arXiv preprint arXiv:2306.06031*, 2023.
- [36] Y. Zhang, W. Chen, F. Yang, X. Zhou, and J. Wang, "Large language models in finance: A survey," *ACM Computing Surveys*, 2024.
- [37] J. S. Park, C. J. O'Brien, C. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the ACM CHI Conference on Human Factors in Computing Systems*, 2023.
- [38] A. Bakhtin, N. B. Wu, S. Gray *et al.*, "Human-level play in the game of diplomacy by combining language models with strategic reasoning," in *Science*, 2022.
- [39] M. L. De Prado, *Advances in financial machine learning*. John Wiley & Sons, 2018.